Introduction to Robotics, Homework Hard 2

Mobile robots: JetBot, Scout 2, Limo CoRoT

Odometry calibration and parking in a pose

MIPT

Center for Cognitive Modeling

What this homework is

Teach a mobile robot to know where it is and to stop where you ask

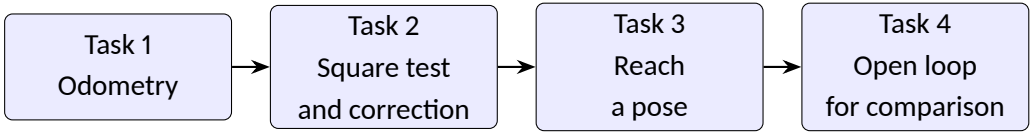
A mobile robot estimates its pose from its wheels, and that estimate is wrong in a way that depends on the particular robot. You measure that error, correct it, and then use the corrected estimate to drive into a marked parking rectangle.

- Choose **one** robot: JetBot (write its number, 1–5), Scout 2, or Limo CoRoT.
- Name the robot on the first page of your report.
- If you chose the Limo arm in Homework Hard 1, use Limo here as well.

The task is stated at a high level. How to do each step, and why it works, is for you to work out and to describe in the report.

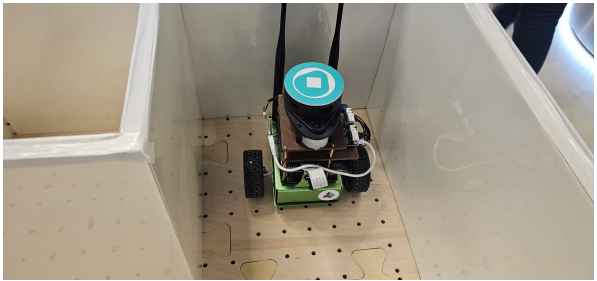
Four tasks, one chain

Each task produces the input of the next one



Odometry (Task 1) is corrected in Task 2 and then feeds the controller of Task 3. Task 4 drives the same goal without any feedback, so you can see what the feedback bought you.

JetBot (units 1--5)

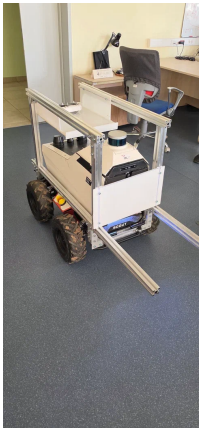


What it is. A small differential-drive robot: two driven wheels and a caster. Each wheel has an encoder with 1760 counts per revolution. The motor board is on a USB serial port. Nominal wheel radius 0.033 m, wheel track 0.15 m.

Your JetBot number. Set ROBOT_ID to it; every log carries it.

Documentation. The header of jetbot_hw.py.

Scout 2 (AgileX)



What it is. A four-wheel skid-steer outdoor robot: it turns by driving the left and right wheels at different speeds, and the wheels slip while it turns. Our unit carries a frame and a lidar that this homework does not use.

Control. Over a CAN bus with the ugv_sdk library; it reports its own linear and angular velocity. See scout_hw.py. Keep the remote controller in your hand and the area free.

Documentation. github.com/westonrobot/ugv_sdk; the SCOUT 2.0 user manual.

Limo CoRoT (AgileX), Ackermann mode



What it is. A four-wheel car-like robot with a steered front axle (Ackermann steering). It cannot turn on the spot: its turning radius has a minimum.

Control. Through the pylimo Python library over a serial port: a speed command and a steering angle; it reports its speed and steering angle. See limo_hw.py. Put the robot in Ackermann mode first.

Documentation. pypi.org/project/pylimo; github.com/agilexrobotics/limo_ros2 for the ROS option.

Starter files

All of it runs without ROS; ROS is allowed

File	Robot	What it gives you
jetbot_hw.py	JetBot	encoder ticks, wheel speed commands
scout_hw.py	Scout 2	reported velocity, velocity commands
limo_hw.py	Limo	reported speed and steering, drive command
hw_log.py	all	CSV log whose first line names the robot

Each file prints the robot's state for three seconds when you run it. Nothing in them knows the true wheel radius, track or wheelbase of your robot: finding the corrections is Task 2.

Task 1: Odometry

From wheel motion to the pose (x, y, θ)

Pose = position (x, y) and heading θ of the robot in a frame fixed to the floor.

- **Differential drive, encoders (JetBot):** tick increments ΔN_L, ΔN_R give wheel paths $\Delta s_{L,R} = 2\pi r\, \Delta N_{L,R}/\mathrm{CPR}$ (r wheel radius, CPR counts per revolution). Then $\Delta s = (\Delta s_R + \Delta s_L)/2$, $\Delta\theta = (\Delta s_R - \Delta s_L)/b$ (b wheel track).
- **Skid-steer, reported velocity (Scout 2):** integrate the reported v (linear) and ω (angular).
- **Ackermann, reported speed and steering (Limo):** integrate $\dot{x} = v\cos\theta$, $\dot{y} = v\sin\theta$, $\dot{\theta} = v\tan\delta/L$ with the reported speed v and steering angle δ; the wheelbase L is yours to measure, and the scale of the reported steering angle is to be checked in Task 2.

Log everything with hw_log.py.

Task 1: What to bring

- Your odometry code and a plot of the path it computes for a short drive.
- The robot driven forward 2 m: tape-measure distance against the odometry.
- The robot turned by a full revolution: what the odometry says.

Task 2: Square test and correction

Find out what the nominal numbers get wrong

Mark a $2\,\mathrm{m} \times 2\,\mathrm{m}$ square on the floor with tape.

- Drive its four sides and four 90° turns, first clockwise, then counter-clockwise, three runs each, by the odometry of Task 1.
- After each run measure with a tape how far the robot ended from where it started.
- From the two sets of errors estimate corrections to the wheel radius and the *effective track* (the track that makes the turn angle right; on a skid-steer robot it is larger than the geometric one). Limo: the wheelbase and a steering factor.
- Apply the corrections and repeat.

Task 2: What to bring

- Photos of the taped square with the robot at the end of a run, tape in frame.
- A table of the end-point errors before and after the correction.
- The correction values and how you obtained them.
- Why the two directions of travel are needed.

Task 3: Reach pose

Same goal and same measurements for every robot

Goal pose (x_g, y_g, θ_g): park in a taped rectangle slightly larger than the robot.

- Drive the goal from the same start five times.
- After each run measure the position error and the heading error against the goal.

The only difference between robots is how the controller is written, because the wheels differ: a differential or skid-steer robot can turn on the spot, an Ackermann robot cannot. The next two frames give the controller of each drive type; use the one that fits your robot.

Task 3: Differential and skid-steer drive

JetBot and Scout 2: feedback in polar coordinates

With ρ the distance to the goal, α the angle between the robot heading and the line to the goal, and β the angle between that line and the goal heading, the controller of lecture 8 is

$$v = k_\rho \, \rho, \qquad \omega = k_\alpha \, \alpha + k_\beta \, \beta,$$

stable for $k_\rho > 0$, $k_\beta < 0$, $k_\alpha - k_\rho > 0$. Choose the gains inside these limits and test them.

Task 3: Ackermann drive

Limo: an arc and a straight line

- Measure the minimum turning radius: drive a full circle at the maximum steering angle and measure its diameter on the floor.
- Choose a goal that an arc followed by a straight line can reach (its final heading is then set by the arc; you may add a second arc). Follow the path by *pure pursuit*: aim at the point of the path a distance L_d ahead, at angle α from the heading, with steering $\delta = \arctan(2L \sin \alpha / L_d)$ (L the wheelbase).

Task 3: What to bring

- The gains (or L_d) and the reason for your choice.
- Five runs: a photo of the final position against the taped rectangle, and the measured position error and heading error of each run.
- Plots of the path the odometry recorded.
- Limo: the measured minimum turning radius.

Task 4: Open loop for comparison

The same goal, with no feedback

Compute from the nominal values a timed sequence of commands. JetBot and Scout 2: turn, drive straight, turn to the goal heading. Limo: it cannot turn on the spot, so a timed sequence of speed and steering commands along your arc and straight line. Send it blindly and run it five times from the same start.

- Measure the final position and heading errors as in Task 3.
- Compare the two sets of errors in one table.

What you hand in

The same rules for all four tasks

One report. The first page names your robot (JetBot with its number) and shows a photo of the setup. For each task:

- what you did and why, with the formulas and numbers;
- photos of the setup and the result, with a ruler or tape in frame wherever a distance is measured;
- screenshots of code, printed values and plots;
- a table of several runs, and what went wrong and what you changed.

Code goes as files; the report quotes only what it discusses.

Deserve “A” grade!

– Oleg Bulichev

bulichev.ov@mipt.ru

@Lupasic

Цифра, 521